

The Definitive Guide to Power Query M

MODULE

9

Query Folding and Performance

Understand how Power Query M translates to native source queries, which operations fold and which break the fold chain, how to diagnose folding in Power BI Desktop, and the architectural strategies that maximise performance across SQL, OData, SharePoint, and other connectors.

Module 9: Query Folding and Performance

Module Contents

■ Module Overview

■ Lesson 1: What Is Query Folding?

- Definition and How It Works
- The M Engine vs the Source Engine
- Why Folding Matters for Performance
- Which Connectors Support Folding

■ Lesson 2: Diagnosing Query Folding

- View Native Query in Power Query Editor
- Right-Click Step Indicators
- Performance Analyzer in Power BI Desktop
- SQL Server Profiler and Extended Events

■ Lesson 3: Operations That Fold

- Column Operations: SelectColumns, RemoveColumns, RenameColumns
- Row Operations: SelectRows, Sort, Distinct
- Table.Group — Aggregation Folding
- Table.Join — Join Folding
- Type Operations and Table.TransformColumnTypes

■ Lesson 4: Operations That Break Folding

- Custom Columns and M-Only Functions
- Table.Buffer and Table.ToList
- Merging from Non-Foldable Sources
- The Folding Break Chain — What Happens Next

■ Lesson 5: Restoring and Preserving Folding

- Ordering Steps for Maximum Folding
- Table.View — Custom Fold Implementation
- Value.NativeQuery — Direct SQL Passthrough
- Splitting Queries at the Fold Boundary

■ Lesson 6: Connector-Specific Folding Behaviour

- SQL Server and Azure SQL
- SharePoint and OData
- Excel, CSV, and Flat Files
- Dataverse and Dynamics 365

■ Lesson 7: Performance Architecture Patterns

- Pattern 1: Push-Down Heavy Logic to SQL
- Pattern 2: Staging Query Architecture
- Pattern 3: Incremental Refresh with Folding
- Pattern 4: Import vs DirectQuery Folding
- Pattern 5: Composite Models and Dual Storage

■ Demonstration 1: Diagnosing a Broken Fold Chain

■ Demonstration 2: Restoring Folding with Query Restructuring

■ Lab 9: Query Folding and Performance

- Exercise 1: Diagnosing Folding Status
- Exercise 2: Reordering Steps to Maximise Folding
- Exercise 3: Value.NativeQuery and Table.View
- Exercise 4: Performance Architecture Review

■ Module Review and Takeaways

Module Overview

Query folding is the single most important performance concept in Power Query. A query that folds executes a fraction of the work of one that does not — the source database processes the filter, sort, and aggregation using its optimised execution engine and returns only the rows needed. A query that does not fold pulls every row from the source into Power Query's M engine, which then applies all transformations in memory. For a 50-million-row fact table, the difference between folding and not folding can mean the difference between a 30-second refresh and a 4-hour one.

This module covers the complete folding picture: what it is, how to diagnose it, which operations fold, which break the fold chain, and — most importantly — the architectural patterns that preserve folding in production models. The connector-specific behaviour section is critical for real-world work: SharePoint folds differently from SQL Server, and OData has its own constraints.

NOTE

This module applies to Power BI Import mode. DirectQuery has its own folding semantics covered in Lesson 7. Folding is not relevant for flat files (CSV, Excel) — those sources have no query engine to push work to.

Before You Begin

- Have access to a SQL Server or Azure SQL database for folding demonstrations.
- Install SQL Server Management Studio or Azure Data Studio to inspect generated SQL.
- Enable Performance Analyzer in Power BI Desktop: View > Performance Analyzer.
- Complete Modules 1–8. Query folding concepts reference `Table.Group`, `Table.Join`, `Table.SelectRows`, and `Table.TransformColumnTypes` from Module 6.

LESSON 1

What Is Query Folding?

After completing this lesson, you will be able to:

- Explain what query folding is and why it matters for Power BI performance.
- Describe how the M engine decides whether to fold or evaluate locally.
- Identify which connector types support query folding.
- Explain the relationship between folding and incremental refresh.

Definition and How It Works

Query folding is the process by which the Power Query engine translates M expressions into the native query language of the data source — most commonly SQL. When a query folds, the data source executes the filter, sort, group, and join operations natively. Only the result rows are transferred to Power BI. When a query does not fold, every row from the source is loaded into Power Query's M engine, which then applies all transformations in memory using the .NET runtime.

Query Folds ✓ — Benefits	Query Does Not Fold ✗ — Costs
All transformations run on the source server	All rows loaded into Power Query memory
Source returns only the rows Power BI needs	M engine applies every transformation locally
Network transfer is minimal	Network transfer is the full dataset
Source query engine optimises execution plan	No source-side optimisation possible
Incremental refresh works correctly	Incremental refresh may silently fail
DirectQuery queries are possible	Large tables cause slow or failed refreshes

Which Connectors Support Folding

Folding is only possible for connectors that have a query engine and a corresponding M-to-native-query translator. Flat files have no query engine and cannot fold. The folding capability of the main connectors:

Connector	Folding	Notes
SQL Server / Azure SQL	Full	All standard operations fold to T-SQL
Azure Synapse Analytics	Full	Full fold to T-SQL; large-scale optimised
Azure Data Lake / Parquet	Partial	Partition pruning and column selection fold
SharePoint Online List	Partial	Filter and select fold via OData; joins do not
OData	Partial	Filter (\$filter), select (\$select), top (\$top) fold
Dataverse	Partial	FetchXML fold for filter/select; aggregations limited
PostgreSQL / MySQL	Full	Full fold to respective SQL dialect
Oracle / Teradata	Full	Full fold to respective SQL dialect
Excel / CSV / Flat File	None	No query engine — all operations in M engine
Web / JSON / XML	None	No query engine — all transformations in M
Python / R scripts	None	Script output is a table — no source query engine

TIP

Incremental refresh in Power BI Service requires that the RangeStart and RangeEnd parameters fold to the source. If your query breaks the fold chain before the date filter step, incremental refresh will silently fall back to full refresh, defeating its entire purpose. Always verify folding when configuring incremental refresh.

LESSON 2

Diagnosing Query Folding

After completing this lesson, you will be able to:

- Use 'View Native Query' to see the SQL generated for a folded step.
- Interpret step indicators in the Applied Steps pane.
- Use Performance Analyzer to measure query and M engine time.
- Use SQL Server Profiler to capture and inspect generated queries.

View Native Query

The most direct way to check whether a step folds is to right-click a step in the Applied Steps pane and select View Native Query. If the option is greyed out, the step does not fold — Power Query cannot translate it to a native query. If it is clickable, a dialog shows the exact SQL (or OData, or FetchXML) that will be sent to the source.

```
-- Example: Generated SQL for a simple filter + column select
-- M code:
-- Source = Sql.Database("server", "db"),
-- Orders = Source[[Schema="dbo",Item="Orders"]][Data],
-- Filtered= Table.SelectRows(Orders, each [Status]="Completed" and [Year]=2024),
-- Cols = Table.SelectColumns(Filtered, {"OrderID", "CustomerID", "Amount", "Status"})

-- Generated T-SQL (visible via View Native Query):
SELECT [OrderID], [CustomerID], [Amount], [Status]
FROM [dbo].[Orders]
WHERE [Status] = 'Completed'
AND [Year] = 2024

-- Power Query pushed the filter and column selection to SQL Server.
-- Only matching rows with those four columns are transferred.
-- If 50M rows exist but 10K match, only 10K rows cross the network.
```

Step Indicators in Applied Steps

In Power Query Editor, each step in the Applied Steps pane has a visual indicator:

Indicator	Meaning
White/no indicator	Step folds to source
Grey background indicator	Step may fold — verify with View Native Query
Yellow warning triangle	Step does NOT fold — evaluated by M engine
View Native Query greyed out	Step definitely does not fold

Performance Analyzer

Performance Analyzer (View > Performance Analyzer > Start Recording) measures time at the visual level but reveals where time is spent: DAX query time is model/DAX performance; Direct query time is source query time (folded SQL); Other time is M engine evaluation (non-folded work).

```
// Performance Analyzer output example for a table visual:
//
// TableVisual – Products by Region
```

```
// DAX query : 45 ms ← model/DAX
// Direct query : 2,340 ms ← SQL execution on source
// Other : 120 ms ← M engine (non-folded transformations)
//
// High "Direct query" time → source-side bottleneck → index or view on SQL side
// High "Other" time → non-folded M work → find the fold break and fix it
// High "DAX query" time → DAX measure complexity → optimise in DAX Studio
```

TIP

For deep folding diagnostics on SQL Server, enable Extended Events or SQL Server Profiler with the 'SQL:BatchCompleted' event. Every SQL statement generated by Power Query appears, including the exact predicates, column lists, and joins. This is the ground truth — more reliable than View Native Query for complex multi-step queries.

LESSON 3

Operations That Fold

After completing this lesson, you will be able to:

- Identify which M table operations fold to SQL.
- Understand which Table.SelectRows conditions fold and which do not.
- Know that Table.Group and Table.Join fold for SQL Server connectors.
- Understand how type operations interact with query folding.

The following operations fold to SQL when applied to a SQL Server or Azure SQL connector source. Folding behaviour may differ for other connectors.

M Operation	Folds?	Notes
Table.SelectColumns(t, cols)	Yes	Becomes SELECT col1, col2 ...
Table.RemoveColumns(t, cols)	Yes	Becomes SELECT all_other_cols
Table.RenameColumns(t, pairs)	Yes	Becomes SELECT col AS alias
Table.SelectRows(t, each [x]=v)	Yes	Becomes WHERE x = 'v' — literal value
Table.SelectRows(t, each [x]>y)	Yes	Becomes WHERE x > y — comparison
Table.SelectRows(each [x]<>null)	Yes	Becomes WHERE x IS NOT NULL
Table.SelectRows + List.Contains	No	List.Contains does not fold on most connectors
Table.Sort(t, col)	Yes	Becomes ORDER BY col ASC/DESC
Table.Distinct(t)	Yes	Becomes SELECT DISTINCT
Table.FirstN(t, n)	Yes	Becomes TOP(n) or FETCH FIRST n ROWS
Table.Group(t, key, aggs)	Yes	Becomes GROUP BY with aggregate functions
Table.Join(t1,k1,t2,k2,kind)	Yes	Becomes JOIN — all six JoinKind values
Table.TransformColumnTypes(t)	Yes	Types inferred in SQL — no extra step
Table.AddColumn(t, name, each [x]+[y])	Partial	Simple arithmetic folds; M functions do not
Table.AddColumn with Text.Upper	No	M text functions do not fold to SQL
Table.AddIndexColumn	No	Row numbering requires M engine
Table.ExpandTableColumn (after NestedJoin)	Yes	Join expansion folds with the join

NOTE

Table.SelectRows conditions fold only when the predicate uses operators that translate directly to SQL: =, <>, <, >, <=, >=, and, or, not, is null, is not null. Calling M functions inside the condition (e.g. Text.Contains, Date.Year, List.Contains) typically breaks folding because there is no direct SQL equivalent.

LESSON 4

Operations That Break Folding

After completing this lesson, you will be able to:

- Identify the M operations and patterns that break the fold chain.
- Understand that once folding breaks, all subsequent steps also do not fold.
- Explain the fold break chain and its cumulative performance impact.
- Recognise patterns that inadvertently break folding.

The Fold Break Chain

Once any step breaks folding, all subsequent steps also execute in the M engine — even if they would otherwise fold. The fold break creates a chain reaction. If a custom column is added at step 5 of 20, steps 6–20 all run in the M engine regardless of whether they use foldable operations. This is why step order is critical: fold-breaking steps must come after all foldable operations.

M Operation	Folds?	Notes
Table.AddColumn with M function	No	Text.Upper, Date.Year, etc. have no SQL equiv.
Table.AddColumn with if/then/else	Partial	Simple conditions may fold; nested rarely do
Table.AddIndexColumn	No	Sequential index requires M engine row order
Table.Buffer(t)	No	Forces full evaluation; severs fold chain
Table.ToList / Table.ToRecords	No	Converts table to list — M engine only
List.Contains in SelectRows	No	No direct SQL IN translation in most connectors
Text.Contains / Text.StartsWith	No	LIKE equivalent not always supported
Date.Year([DateCol]) = 2024	No	Date function not directly foldable; use date range
Joining to a non-foldable source	No	Cross-source join always breaks folding
Table.FillDown / Table.FillUp	No	Window function equivalent not supported
Table.TransformColumns with fn	No	Custom function per cell breaks fold
Custom M function in column	No	M functions have no SQL equivalent
Accessing relative rows (t{n})	No	Positional row access is M-only
List.Generate inside a query	No	Iteration is M-only

The Most Common Inadvertent Fold Breaks

```
// ■ BREAK 1: Date function in filter – breaks folding
// This does NOT fold – Date.Year is an M function
Table.SelectRows( Orders, each Date.Year([OrderDate]) = 2024 )

// ✓ FIX: Use a date range – this FOLDS to WHERE OrderDate >= '2024-01-01'
Table.SelectRows( Orders,
```

```
each [OrderDate] >= #date(2024,1,1) and [OrderDate] < #date(2025,1,1) )

// ■ BREAK 2: List.Contains in filter – breaks folding on most connectors
ValidRegions = {"East","West","South"},
Table.SelectRows( Sales, each List.Contains(ValidRegions, [Region]) )

// ✓ FIX: Use OR conditions – folds to WHERE Region IN ('East','West','South')
Table.SelectRows( Sales,
each [Region] = "East" or [Region] = "West" or [Region] = "South" )
// For large lists: push to source via a JOIN instead

// ■ BREAK 3: AddIndexColumn before SelectRows – index breaks fold, filter does not
Indexed = Table.AddIndexColumn(Source, "RowNum", 1, 1),
Filtered = Table.SelectRows(Indexed, each [Status] = "Active") // now does not fold!

// ✓ FIX: SelectRows BEFORE AddIndexColumn
Filtered = Table.SelectRows(Source, each [Status] = "Active"), // folds ✓
Indexed = Table.AddIndexColumn(Filterd, "RowNum", 1, 1) // M engine – but fewer rows

// ■ BREAK 4: Sorting before a non-foldable step – wasted sort
Sorted = Table.Sort(Source, "Date"), // folds ✓
WithCols = Table.AddColumn(Sorted, "Tag", each Text.Upper([Name])) // breaks fold
// Sort is re-done in M engine anyway – the SQL ORDER BY was pointless

// ✓ FIX: Put non-foldable operations after ALL foldable ones
Filtered = Table.SelectRows(Source, each [Status] = "Active"),
Grouped = Table.Group(Filterd, "Region", {"Total", each List.Sum([Amount])}),
WithCols = Table.AddColumn(Grouped, "Label", each Text.Upper([Region]))
```

LESSON 5

Restoring and Preserving Folding

After completing this lesson, you will be able to:

- Reorder query steps to move fold-breaking operations to the end.
- Use `Value.NativeQuery` to pass raw SQL directly to the source.
- Understand the purpose and pattern of `Table.View` for custom folding.
- Split queries at the fold boundary to contain M engine work.

The Golden Rule: Foldable Operations First

The single most impactful performance practice in Power Query is: perform all foldable operations first — before any operation that breaks folding. The ideal query structure is:

```
// Ideal query structure: foldable block → fold break → M-only block
let
// ■■■ FOLDABLE BLOCK – all of this becomes one SQL statement ■■■■■■■■■■
Source = Sql.Database("server", "AdventureWorks"),
Table = Source[Schema="Sales",Item="SalesOrderHeader"][Data],
Filtered = Table.SelectRows(Table, each [Status]=5 and [OrderDate]>=#date(2024,1,1)),
Cols = Table.SelectColumns(Filterd, {"SalesOrderID","OrderDate","CustomerID",
"TotalDue","TerritoryID"}),
Joined = Table.Join(Cols,"TerritoryID", SalesTerritories,"TerritoryID",JoinKind.LeftOuter),
Grouped = Table.Group(Joined,"TerritoryID",{{"Revenue",each List.Sum([TotalDue]),type number}}),
// ↑ All above → single SQL with WHERE, SELECT, JOIN, GROUP BY, SUM

// ■■■ FOLD BREAK – beyond this point, M engine handles everything ■■■■■
WithLabel = Table.AddColumn(Grouped, "TerritoryLabel",
each Text.Upper([Name]) & " (" & Text.From([TerritoryID]) & ")", type text),
WithIndex = Table.AddIndexColumn(WithLabel, "Rank", 1, 1, Int64.Type),

// ■■■ M-ONLY BLOCK – fine because we're working on the small result ■■■
Sorted = Table.Sort(WithIndex, {{"Revenue",Order.Descending}}),
Final = Table.TransformColumnTypes(Sorted,
{{"Revenue",type number},{ "Rank",Int64.Type}})
in
Final
// The SQL query pulls ~10K rows; the M engine works on those 10K, not the full 50M table
```

Value.NativeQuery — Direct SQL Passthrough

```
// Value.NativeQuery passes a raw SQL statement directly to the source
// Useful when M cannot express the required SQL, or when a stored procedure is needed

let
Source = Sql.Database("server", "AdventureWorks"),
Result = Value.NativeQuery(Source,
"SELECT c.CustomerID, c.AccountNumber, SUM(h.TotalDue) AS Revenue
FROM Sales.Customer c
JOIN Sales.SalesOrderHeader h ON c.CustomerID = h.CustomerID
WHERE h.OrderDate >= '2024-01-01'
GROUP BY c.CustomerID, c.AccountNumber
ORDER BY Revenue DESC",
null,
[ EnableFolding = true ] // allow further M folding on top of this result
)
```

```

in
Result

// Value.NativeQuery with parameters (safe – no SQL injection)
Value.NativeQuery(Source,
"SELECT * FROM Sales.Orders WHERE Status = @status AND Year = @year",
[ status = "Active", year = 2024 ]
)

```

WARNING

Value.NativeQuery bypasses all Power Query safety checks. The SQL is sent directly to the source as-is. Only use parameterised queries (using the third argument) — never concatenate user inputs into the SQL string. Parameterised queries prevent SQL injection. Also note that EnableFolding=true allows M to add further WHERE conditions on top — used for incremental refresh.

Splitting Queries at the Fold Boundary

```

// Pattern: split at the fold boundary into a staging query + transformation query
// StagingQuery (fully foldable – stored as a separate Power Query query):
let
Source = Sql.Database("server", "AdventureWorks"),
Orders = Source[Schema="Sales", Item="SalesOrderHeader"][Data],
Filtered = Table.SelectRows(Orders, each [OrderDate] >= RangeStart and [OrderDate] < RangeEnd),
Joined = Table.Join(Filtered, "TerritoryID", SalesTerritories, "TerritoryID", JoinKind.LeftOuter),
Grouped = Table.Group(Joined, "TerritoryID", {"Revenue", each List.Sum([TotalDue]), type number})
in
Grouped // ← this query is fully foldable → single SQL statement

// TransformationQuery (M-only, operates on small grouped result):
let
Staged = Table.Buffer(StagingQuery), // buffer the staged result
WithLabel = Table.AddColumn(Staged, "Label", each Text.Upper([Name]), type text),
WithRank = Table.AddIndexColumn(WithLabel, "Rank", 1, 1, Int64.Type),
Sorted = Table.Sort(WithRank, {"Revenue", Order.Descending})
in
Sorted

// Benefit: the staging query folds and returns a small result.
// The transformation query runs M engine only on the small result.
// Power BI Service only needs to refresh the staging query for incremental refresh.

```

LESSON 6

Connector-Specific Folding Behaviour

After completing this lesson, you will be able to:

- Understand the specific folding capabilities of SQL Server, SharePoint, OData, and Dataverse.
- Know which operations fold for SharePoint lists and which do not.
- Apply the correct folding patterns for OData \$filter, \$select, and \$top.
- Identify folding limitations in Dataverse and Dynamics 365 connectors.

SQL Server and Azure SQL — Full Folding

SQL Server and Azure SQL provide the most complete folding support. All standard operations — filter, project, sort, group, join, distinct, top — translate to T-SQL. The generated SQL is visible via View Native Query. Best practices specific to SQL Server folding:

```
// ✓ Date range folds – use this instead of Date.Year()
Table.SelectRows( Orders, each [OrderDate] >= #date(2024,1,1) and [OrderDate] < #date(2025,1,1) )
// → WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01'

// ✓ Table.Group folds completely to GROUP BY + aggregate functions
Table.Group(Sales, "Region", {
  {"Revenue", each List.Sum([Amount]), type number},
  {"Count", each Table.RowCount(_), Int64.Type}
})
// → SELECT Region, SUM(Amount) AS Revenue, COUNT(*) AS Count FROM Sales GROUP BY Region

// ✓ Stored procedures via Value.NativeQuery
Value.NativeQuery(Sql.Database("srv","db"), "EXEC usp_GetSales @year=@y", [y=2024])

// ✗ BREAKS: Table.AddColumn with Text.Contains or custom function
// ✗ BREAKS: Table.SelectRows with List.Contains
// ✗ BREAKS: Table.AddIndexColumn anywhere in the fold block
```

SharePoint Online Lists — Partial Folding via OData

```
// SharePoint folds via OData protocol – limited to $filter, $select, $top, $orderby
// The M function SharePoint.Tables or SharePoint.Contents generates OData queries

// ✓ FOLDS: Simple equality and comparison filters
Table.SelectRows(spList, each [Status] = "Active")
// → $filter=Status eq 'Active'

// ✓ FOLDS: Column selection
Table.SelectColumns(spList, {"Title","Status","Created","Modified"})
// → $select=Title,Status,Created,Modified

// ✓ FOLDS: Table.FirstN (top N rows)
Table.FirstN(spList, 5000)
// → $top=5000

// ✗ DOES NOT FOLD: Joins – SharePoint has no cross-list join in OData
Table.Join(spList, "LookupID", anotherList, "ID", JoinKind.LeftOuter)
// Must join in M engine – loads both lists in full

// ✗ DOES NOT FOLD: Table.Group – SharePoint OData does not support $apply aggregation
// ✗ DOES NOT FOLD: Text.Contains – no contains() in SharePoint OData dialect
```

```
// Performance tip: SharePoint lists cap at 5000 rows per OData request by default.  
// Use list view threshold settings and indexed columns to enable larger filtered requests.
```

OData Feeds — Standard Partial Folding

```
// OData connectors fold to standard OData query options  
// Supported: $filter, $select, $top, $skip, $orderby, $count  
  
// ✓ FOLDS: Standard comparisons  
Table.SelectRows(feed, each [Year] = 2024 and [Country] = "UK")  
// → $filter=Year eq 2024 and Country eq 'UK'  
  
// ✓ FOLDS: Top N (maps to $top)  
Table.FirstN(feed, 100)  
// → $top=100  
  
// ✗ NOT SUPPORTED in standard OData: GROUP BY ($apply requires OData v4 + server support)  
// ✗ NOT SUPPORTED: JOIN across entities (use $expand for navigation properties)  
  
// OData $expand for navigation properties (related entities)  
OData.Feed("https://services.odata.org/V4/OData/OData.svc/Products?$expand=Category")
```

LESSON 7

Performance Architecture Patterns

After completing this lesson, you will be able to:

- Apply the staging query architecture to maximise folding in complex models.
- Configure incremental refresh with correct fold-preserving query structure.
- Understand the performance differences between Import and DirectQuery modes.
- Apply composite model strategies to balance performance and freshness.

Pattern 1: Push Heavy Logic to SQL

```
// Pattern: move complex joins and aggregations into a SQL view, not M
-- In SQL Server, create a view:
CREATE VIEW dbo.vw_SalesSummary AS
SELECT
  t.TerritoryName,
  p.ProductCategory,
  YEAR(o.OrderDate) AS OrderYear,
  MONTH(o.OrderDate) AS OrderMonth,
  SUM(d.LineTotal) AS Revenue,
  COUNT(DISTINCT o.SalesOrderID) AS OrderCount
FROM Sales.SalesOrderHeader o
JOIN Sales.SalesOrderDetail d ON o.SalesOrderID = d.SalesOrderID
JOIN Production.Product p ON d.ProductID = p.ProductID
JOIN Sales.SalesTerritory t ON o.TerritoryID = t.TerritoryID
WHERE o.Status = 5
GROUP BY t.TerritoryName, p.ProductCategory, YEAR(o.OrderDate), MONTH(o.OrderDate)

-- Then in M: simply load the view – one foldable step
let
  Source = Sql.Database("server","AdventureWorks"),
  Summary = Source[Schema="dbo",Item="vw_SalesSummary"][Data]
in
  Table.TransformColumnTypes(Summary,
    {"Revenue",type number},{ "OrderYear",Int64.Type},{ "OrderCount",Int64.Type})
```

Pattern 2: Incremental Refresh with Folding

```
// Incremental refresh REQUIRES RangeStart and RangeEnd to fold to the source
// These are Power BI Service parameters injected at refresh time

let
  Source = Sql.Database("server","AdventureWorks"),
  Orders = Source[Schema="Sales",Item="SalesOrderHeader"][Data],

  // RangeStart and RangeEnd MUST be in a Table.SelectRows that folds
  // Do NOT process them with M date functions first – that breaks folding
  Filtered = Table.SelectRows(Orders,
    each [OrderDate] >= RangeStart and [OrderDate] < RangeEnd),
  // ↑ Both parameters fold directly to SQL WHERE clause

  // Foldable operations continue after the date filter
  Typed = Table.TransformColumnTypes(Filtered,
    {"OrderDate",type date},{ "TotalDue",type number}),

  // ONLY add non-foldable steps after the foldable block
```

```

WithYear = Table.AddColumn(Typed, "Year", each Date.Year([OrderDate]), Int64.Type)
in
WithYear

// WRONG – this breaks folding for incremental refresh:
// Dated = Table.AddColumn(Source, "Year", each Date.Year([OrderDate])), // breaks fold!
// Filtered = Table.SelectRows(Dated, each [Year] = ... ) // RangeStart never reaches SQL

```

Pattern 3: Import vs DirectQuery Folding Comparison

Import mode loads data into Power BI's VertiPaq engine at refresh. DirectQuery sends a query to the source on every visual interaction. Both benefit from query folding, but the impact is different:

Import Mode — Folding Impact	DirectQuery Mode — Folding Impact
Folding reduces rows loaded at refresh	Every visual interaction generates a SQL query
Poor folding increases refresh time by minutes	Folding is critical for every interaction response time
Non-folded steps run once at refresh	Non-folded steps run on every click
VertiPaq stores compressed data in RAM	Source must handle concurrent query load
Visual interactions are instant (RAM)	Latency depends on source response time

TIP

For large tables that require near-real-time data, consider a Composite Model: import the historical fact table partition (folded, fast refresh) and use DirectQuery only for the current-day partition. This gives the speed of import for most data and the freshness of DirectQuery for today's transactions — without paying the DirectQuery latency for every interaction.

DEMONSTRATION 1

Diagnosing a Broken Fold Chain

Scenario: A Power BI model refreshes slowly. A query against a 20-million-row SQL Server fact table takes 8 minutes. The task is to diagnose where the fold chain breaks and identify the cause.

Demonstration Steps

1	Open the slow query in Power Query Editor. Count the Applied Steps — there are 14 steps. Right-click each step and attempt to select View Native Query. Note which steps have it greyed out.
2	Identify the first step where View Native Query becomes greyed out — this is the fold break. In this example: step 6 is Table.AddColumn with Text.Upper([CustomerName]).
3	Steps 7–14 (SelectRows, Sort, Group) all come AFTER the fold break. None of them fold, even though they are individually foldable operations. Show the SQL Server Profiler trace — only one SQL SELECT * is sent to SQL Server. All 20M rows are pulled into the M engine.
4	Calculate the cost: 20M rows × 12 columns × ~50 bytes = ~12 GB transferred. The M engine sorts and groups 20M rows in RAM. This is why it takes 8 minutes.
5	Fix step 1: Move Text.Upper to after all foldable operations. Reorder steps so SelectRows (year filter), Sort, and Group come before AddColumn.
6	Verify: right-click steps 1–8. All show View Native Query — all fold. Show the new SQL: SELECT with WHERE, ORDER BY, GROUP BY, SUM. SQL Server returns 1,200 rows. The M engine only works on 1,200 rows.
7	Measure: refresh time drops from 8 minutes to 22 seconds.

RESULT

A single step reordering reduces refresh time from 8 minutes to 22 seconds — a 95% reduction. The fold break was caused by a custom column placed before all filtering and aggregation steps.

DEMONSTRATION 2

Restoring Folding with Query Restructuring

Scenario: A query uses `Date.Year([OrderDate]) = 2024` as its row filter, which breaks folding. The task is to replace it with a foldable date range, then verify folding is restored using View Native Query and SQL Profiler.

Demonstration Steps

1	Show the original query: <code>Table.SelectRows(Source, each Date.Year([OrderDate])=2024)</code> . Right-click — View Native Query is greyed out. SQL Profiler shows: <code>SELECT * FROM Sales.Orders</code> (no WHERE clause — all 50M rows transferred).
2	Replace with: <code>Table.SelectRows(Source, each [OrderDate] >= #date(2024,1,1) and [OrderDate] < #date(2025,1,1))</code> . Right-click — View Native Query is now active.
3	Show the generated SQL: <code>SELECT ... FROM Sales.Orders WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01'</code> . SQL Server returns 2.1M rows instead of 50M.
4	Apply the same fix to a <code>List.Contains</code> filter: replace each <code>List.Contains({"East","West"}, [Region])</code> with <code>each [Region]="East" or [Region]="West"</code> . Verify both now fold.
5	Add a <code>Table.Group</code> step after the filter. Show it also folds — the generated SQL now has both WHERE and GROUP BY. SQL Server returns 8 rows.
6	Introduce <code>Table.AddColumn</code> with <code>Text.Upper</code> after Group. Show fold breaks here but only the AddColumn step is in M engine — working on 8 rows, not 50M.
7	Compare Performance Analyzer before and after: 'Direct query' time: 18s → 0.4s. 'Other' (M engine) time: negligible.

RESULT

Two filter replacements restore folding and reduce the source query from a full table scan to a WHERE + GROUP BY returning 8 rows. Refresh time for this query drops by over 97%.

LAB 9

Query Folding and Performance

In this lab you diagnose and fix query folding in a Power BI model connecting to SQL Server (or the AdventureWorks sample database). You will identify fold breaks, reorder steps, apply Value.NativeQuery, and configure correct incremental refresh structure.

Lab Scenario

The Contoso BI team reports that the nightly refresh takes over 3 hours. Your task is to audit all major queries for folding issues, fix the worst offenders, and implement the staging query architecture for the two largest fact tables.

NOTE

Use SQL Server with AdventureWorksDW or AdventureWorks sample databases. Enable SQL Server Profiler or Extended Events to capture generated SQL. Enable View Native Query in Power Query by connecting via `Sql.Database()`, not ODBC.

Exercise 1: Diagnosing Folding Status (Estimated time: 20 minutes)

- Connect to AdventureWorksDW. Load FactInternetSales, DimCustomer, and DimProduct.
- For FactInternetSales: right-click each Applied Step and record which fold (green) and which do not (greyed out View Native Query). Identify the exact step that first breaks the fold chain.
- Enable SQL Profiler. Trigger a refresh of FactInternetSales. Observe the SQL statement(s) sent to SQL Server. How many rows are returned? Is there a WHERE clause?
- Open Performance Analyzer. Refresh a table visual using FactInternetSales. Record: Direct query time, Other time, DAX query time.
- Identify all fold-breaking patterns in the query: `Date.Year()` in filters, `List.Contains`, `AddIndexColumn`, or custom M functions.
- Document findings: create a table listing each broken step, its cause, and the proposed fix.

RESULT

A complete fold audit showing which steps fold, which do not, and the root cause of each fold break. SQL Profiler confirms how many rows are transferred before and after each fold break.

Exercise 2: Reordering Steps to Maximise Folding (Estimated time: 25 minutes)

- Fix the date filter: replace `Date.Year([OrderDateKey])=2020` with a date range filter using `#date()` literals. Verify the fold is restored with View Native Query.
- Fix the `List.Contains` filter: replace `List.Contains(validKeys,[ProductKey])` with explicit OR conditions or a JOIN to a dimension table. Verify folding.
- Move `Table.AddIndexColumn` and all custom M function columns to after all foldable steps. Verify that steps 1 through N now all show View Native Query.
- Run SQL Profiler again. Show the new generated SQL: it should contain WHERE, GROUP BY, and JOINS — all in one statement. Count the rows returned.
- Measure the Performance Analyzer improvement: record Direct query and Other times. Calculate the percentage improvement.

- **STRETCH:** Apply the staging query split pattern. Create StagingQuery (fully foldable) and TransformationQuery (M-only, works on small staged result). Verify StagingQuery shows View Native Query on every step.

RESULT

All foldable operations now fold. SQL Profiler shows a single optimised SQL statement with WHERE, GROUP BY, and JOINS. Row count transferred is reduced by at least 80%.

Exercise 3: Value.NativeQuery and Advanced Folding (Estimated time: 20 minutes)

- Write a Value.NativeQuery call that executes a stored procedure: EXEC usp_GetMonthlySales @Year=2024. Cast the result to a typed table in M.
- Write a Value.NativeQuery with parameterised inline SQL: SELECT * FROM dbo.FactInternetSales WHERE OrderDateKey BETWEEN @start AND @end. Pass the start and end values as record parameters.
- Enable EnableFolding=true on the Value.NativeQuery call. Add a Table.SelectRows step on the result. Verify it folds (the WHERE is appended to the SQL).
- Compare performance: Value.NativeQuery vs equivalent M steps — measure refresh time for both approaches on FactInternetSales.
- **STRETCH:** Implement Table.View on a flat file source to add simulated folding. Override the OnSelectRows handler to filter the underlying list before building the table.

RESULT

Value.NativeQuery with parameters executes correctly and safely. EnableFolding=true allows further M conditions to be appended to the native SQL. Performance is measured and compared.

Exercise 4: Performance Architecture Review (Estimated time: 25 minutes)

- Review the full Contoso model. Identify the three queries with the highest refresh time from Performance Analyzer. Confirm whether folding is active for each.
- For the largest fact table: implement the SQL view pattern (Pattern 1). Create a SQL view that pushes the join and aggregation to SQL Server. Replace the M join+group with a single view load.
- Configure incremental refresh on FactInternetSales: set RangeStart and RangeEnd parameters. Verify that the date filter uses these parameters directly in Table.SelectRows without any M date function wrapping.
- Verify incremental refresh folding: right-click the filtered step — it must show View Native Query. If greyed out, find and fix the fold break.
- Load the model to Power BI Service. Trigger a manual refresh. Check the refresh history — confirm incremental refresh ran (not full refresh).
- **STRETCH:** Convert the largest dimension table from Import to Dual storage. Explain how Dual storage benefits both Import and DirectQuery queries on the fact table.

RESULT

The three slowest queries are fixed. Incremental refresh is correctly configured and verified as folding. The SQL view reduces M engine work to zero for the largest fact table. Estimated refresh time reduction: 70–90%.

Module Review and Takeaways

Review Questions

What is query folding and why does it matter?

Answer: Query folding is the process by which Power Query translates M expressions into the native query language of the data source (e.g. T-SQL). When folding occurs, the source engine processes filters, sorts, groups, and joins — returning only the needed rows. Without folding, every row is transferred to the M engine, which applies all transformations in memory. For large tables, folding can reduce refresh time by 90%+ and is required for incremental refresh to work correctly.

Once a fold break occurs at step 5 of 20, do steps 6–20 fold?

Answer: No. Once any step breaks the fold chain, all subsequent steps also run in the M engine — even if they are individually foldable operations. This is the fold break chain. The M engine processes every step from the fold break onward on the full dataset that was returned from the source. This is why the position of fold-breaking steps is critical: they must come after all foldable operations.

Why does `Date.Year([OrderDate]) = 2024` break folding, and how do you fix it?

Answer: `Date.Year` is an M function with no direct SQL equivalent — the connector cannot translate it to a SQL WHERE clause. The fix is to use a date range with M date literals that translate directly to SQL comparison operators: `[OrderDate] >= #date(2024,1,1)` and `[OrderDate] < #date(2025,1,1)` folds to `WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01'`.

How do you diagnose whether a step folds?

Answer: Right-click the step in Applied Steps and select View Native Query. If it is greyed out, the step does not fold. If it is clickable, it shows the exact SQL sent to the source. Additionally, SQL Server Profiler or Extended Events captures every SQL statement generated by Power Query — this is the ground truth for complex multi-step queries. Performance Analyzer shows 'Direct query' vs 'Other' time to identify whether the M engine is doing significant work.

What is `Value.NativeQuery` and when should you use it?

Answer: `Value.NativeQuery` passes a raw SQL statement directly to the source, bypassing M's translation layer. Use it when: M cannot express the required SQL logic, a stored procedure must be called, or a complex analytical query is best written in SQL. Always use parameterised queries (third argument) — never concatenate inputs into the SQL string. With `EnableFolding=true`, further M conditions can be appended to the native query.

Why is correct incremental refresh structure important for query folding?

Answer: Power BI Service injects `RangeStart` and `RangeEnd` parameters to filter each incremental partition. These parameters must fold directly to the source — they must appear in a `Table.SelectRows` condition without any M function wrapping. If the fold chain is broken before the date filter, Power BI falls back to full refresh silently, defeating incremental refresh entirely.

What are the folding limitations of SharePoint Online lists?

Answer: SharePoint folds via OData: simple equality/comparison filters (`$filter`), column selection (`$select`), and top N (`$top`) fold. `Table.Join` does NOT fold — SharePoint OData cannot join across lists; both lists are loaded in full into the M engine. `Table.Group` does not fold. `Text.Contains` does not fold. SharePoint lists also have a 5000-row default view threshold that limits unfiltered list access.

What is the staging query architecture and why is it useful?

Answer: The staging query architecture splits a query at the fold boundary: a staging query contains all foldable operations and produces a small result set; a transformation query loads the staged result and applies M-only operations. Benefits: the staging query folds completely to the source; the transformation query runs the M engine only on the small staged result; incremental refresh can target the staging query; and the separation makes the query easier to maintain.

Best Practices

- Apply all foldable operations first — filter, join, group, project — before any operation that breaks folding. The fold break chain means step order directly determines how many rows the M engine must process.
- Always replace `Date.Year([col]) = year` conditions with date range literals: `[col] >= #date(year,1,1)` and `[col] < #date(year+1,1,1)`. Date function calls never fold on any connector.
- Use View Native Query on every step after adding a new transformation. It takes 10 seconds and immediately reveals if you have inadvertently broken folding.
- For incremental refresh: the RangeStart/RangeEnd filter must fold. Test by right-clicking the date filter step — if View Native Query is greyed out, incremental refresh will silently fall back to full refresh.
- Use `Value.NativeQuery` for complex joins and aggregations that are better expressed in SQL. Always parameterise — never concatenate values into the SQL string.
- Use the staging query split pattern for any query that must mix foldable and non-foldable operations. The staging query returns a small result set; the transformation query works on that small result.
- For SharePoint lists, always apply column selection (`Table.SelectColumns`) and row filtering (`Table.SelectRows` with simple conditions) immediately after loading the source — before any joins or custom columns.
- Push heavy joins and aggregations into SQL views when possible. A view load in M is a single foldable step — the most efficient M query possible.
- Use Performance Analyzer to identify the actual bottleneck before optimising. High 'Direct query' time means the source is slow — add indexes or rewrite SQL. High 'Other' time means the M engine is working too hard — find the fold break.
- For OData sources, test which operations actually fold — OData server implementations vary. Some support \$apply (group by), most do not. Always verify with actual queries.

Common Issues and Solutions

View Native Query is greyed out on a step that should fold

Cause: A previous step broke the fold chain. Even if the current step uses a foldable operation, it runs in the M engine because an earlier fold break makes everything after it M-only.

Solution: Work backwards through Applied Steps using View Native Query on each step. Find the first step where it becomes greyed out — that is your fold break. Fix that step.

Incremental refresh is running as full refresh in Power BI Service

Cause: The RangeStart/RangeEnd filter does not fold to the source. This happens when the date parameters are processed with M functions (`Date.Year`, `Date.Month`) before being used in `Table.SelectRows`, or when a fold break occurs before the date filter step.

Solution: Ensure the filter is: `Table.SelectRows(source, each [Date] >= RangeStart and [Date] < RangeEnd)` with no M function wrapping. Right-click that specific step in Desktop — View Native Query must be active.

Formula.Firewall error when combining a SQL source with another source

Cause: Power BI's privacy firewall blocks combining data from sources with different privacy levels in a single query (e.g. SQL Server + local Excel config file).

Solution: Set privacy levels for all data sources: either set all to Organisational/Public, or in Desktop only (not Service), disable privacy checks in Options > Privacy. For production, use a parameterised SQL query to

avoid the cross-source join in M.

SharePoint list returns only 5000 rows (list view threshold)

Cause: SharePoint Online enforces a 5000-item view threshold for unfiltered list access. Unindexed columns in filters also trigger this limit.

Solution: Ensure all filter columns are indexed in the SharePoint list settings. Apply `Table.SelectRows` with indexed column conditions before any join or custom column. For lists over 100K items, consider exporting to SQL Server or Azure SQL instead.

Value.NativeQuery fails with 'Native query not supported'

Cause: The connector does not support native queries, or the connection is using an ODBC driver rather than the native connector.

Solution: Use `Sql.Database()` for SQL Server (not ODBC). For other databases, check connector documentation for native query support. Some connectors require `EnableFolding=true` in the options record to enable `Value.NativeQuery`.

Key Concepts — Module 9 Quick Reference

Folding status reference for the most common M operations against SQL Server.

M Operation	Folds?	Notes
Table.SelectColumns / RemoveColumns	Yes	Becomes SELECT col list
Table.SelectRows – literal values	Yes	Becomes WHERE col = 'value'
Table.SelectRows – date range (#date)	Yes	Becomes WHERE col >= date AND col < date
Table.SelectRows – Date.Year()	No	M function — no SQL equivalent
Table.SelectRows – List.Contains	No	No direct SQL IN translation
Table.Sort	Yes	Becomes ORDER BY
Table.Distinct	Yes	Becomes SELECT DISTINCT
Table.FirstN	Yes	Becomes TOP(n) / FETCH FIRST n
Table.Group – List.Sum/Count/Avg	Yes	Becomes GROUP BY + SUM/COUNT/AVG
Table.Join – all JoinKind values	Yes	Becomes JOIN type
Table.TransformColumnTypes	Yes	Type cast handled by SQL connector
Table.AddColumn – arithmetic	Partial	Simple arithmetic may fold
Table.AddColumn – M functions	No	Text.Upper, Date.Year etc. do not fold
Table.AddIndexColumn	No	Sequential index requires M engine
Table.Buffer	No	Forces full evaluation; severs fold chain
Table.FillDown / FillUp	No	Window function equivalent not supported
Value.NativeQuery	Direct	Raw SQL passthrough to source

Module Summary

- Query folding translates M expressions to native source queries. When queries fold, the source engine does the work and returns only needed rows. Without folding, all rows are transferred to the M engine — dramatically slower for large tables.
- Diagnose folding with View Native Query (greyed out = no fold), SQL Profiler (see actual SQL sent), and Performance Analyzer (Direct query vs Other time). Always verify folding after adding new steps.
- Foldable operations: SelectColumns, SelectRows (simple predicates), Sort, Distinct, FirstN, Group, Join, TransformColumnTypes. These fold to SQL when applied to SQL sources.
- Fold-breaking operations: M functions in predicates (Date.Year, Text.Contains, List.Contains), AddIndexColumn, Buffer, FillDown, and cross-source joins. Once the fold chain breaks, all subsequent steps also run in the M engine.
- Restore folding by reordering steps (foldable last), replacing M functions with native SQL patterns (date ranges instead of Date.Year), and using Value.NativeQuery for complex SQL that cannot be expressed in M.
- Connector-specific: SQL Server has full folding; SharePoint folds filter/select/top via OData but not joins or groups; flat files have no folding.
- Five architecture patterns: push-down to SQL view, staging query split, incremental refresh with folding, Import vs DirectQuery trade-offs, and Composite Models for balanced performance and freshness.

NOTE

Next Module — Module 10: Performance Optimisation and Advanced Architecture. Covers VertiPaq storage, DAX/M interaction, Table.Buffer strategy, column cardinality and its impact on model size, DirectQuery optimisation, star schema construction in M, and the full set of Power BI refresh performance techniques.